

LLD Notes 01 — Why Software Design Exists

Dependencies, Inversion of Control, Coupling & Cohesion, and SOLID

Theme: Coding is easy. Engineering software that survives time is hard.

1. The Big Picture

Writing 1,000 lines of fresh code is easy — there is no history, no consumers, no hidden expectations. Changing 1 line in a million-line codebase is hard — you must first understand everything that *depends* on that line. This is the entire difference between **programming** (writing instructions that work today) and **software engineering** (designing for changes that don't exist yet). Companies pay engineers because **requirements change**; if they never changed, you'd be paid once and let go. The industry also carries a structural handicap: the population of programmers has doubled roughly every five years since the 1960s, which mathematically means **half of all working programmers always have less than five years of experience** — design wisdom must therefore be deliberately taught, it cannot be absorbed through mentorship alone. And in the AI era this matters *more*, not less: AI has solved *writing* code, but it has not solved *changing* code safely. AI tools bill by context, and context is a function of coupling — in a well-designed codebase a change touches 2 files and needs a few thousand tokens; in a tangled one the same change touches 40 files, needs enormous context, and still breaks tests. Coupling is no longer just a maintenance cost; it is a literal dollar cost per prompt. AI reduced the value of typing and increased the value of thinking — and "thinking," in software, largely means deciding **where code lives and which direction the dependency arrows point**. Finally, remember the reason we write bad code in the first place: "we had to go fast." But you don't go fast by rushing — every future touch of rushed code pays a tax. A surgeon operating on *you*, under a deadline, should still behave carefully and deliberately. So should we.

2. Symptoms of Bad Code

Good code is **boring** — you read it and think "obviously." Bad code has three named diseases:

2.1 Rigidity

One change forces many other changes.

You know exactly where a bug is. You estimate "3 weeks." You open the module — but fixing it means touching a caller, which means touching *its* callers, and the chain never ends.

Wanted to change: [A]
Forced to change: [A]→[B]→[C]→[D]→ ... every module in the system

Rigidity = dependencies snake out in so many directions that no change can be isolated.

2.2 Fragility

Change one place → things break in *unrelated* places.

You change how salary is calculated for hourly employees — and the union report starts crashing. Like a mechanic who fixes your electric window and now your engine won't start: you conclude he's incompetent, and you never go back. That is exactly how managers perceive us. The ultimate failure is when management declares "*nobody touches that module*" — the business, who cannot even read code, has correctly diagnosed that the engineers lost control.

2.3 Immobility

Useful code that cannot be reused because it drags its whole world along.

Joe's module does exactly what you need — but it's welded to a bizarre framework, a weird database, and three global singletons. Extracting the useful part costs more than rewriting it. **The desirable parts are coupled to the undesirable parts.**

2.4 The common cause

All three diseases have a single root: **coupling** — dependencies running in strange and undesirable directions.

The thesis of this course: "The bulk of software design is managing dependencies — deciding where code lives, and which direction the arrows point." Every design pattern you will ever learn is just another way of controlling dependencies.

3. Coupling and Cohesion

These two words are the vocabulary of software design. Every principle you learn later (SOLID, GRASP, Clean Architecture) is ultimately in service of one goal:

LOW COUPLING + HIGH COHESION

3.1 Coupling — how entangled modules are with each other

Coupling measures how much one module must know about another to work. High coupling means a change in one module ripples into others.

Levels of coupling, from worst to best:

Level	Meaning	Example
Content coupling (worst)	One module reaches into another's internals and modifies them	<code>order.items.append(x)</code> from outside the Order class
Common coupling	Modules share mutable global state	Global config object everyone reads AND writes
Control coupling	One module passes a flag telling another <i>how</i> to behave	<code>save(data, useCache=True, retry=False, mode=3)</code>
Stamp coupling	Passing a big object when only one field is needed	<code>printName(employee)</code> when only the name is used
Data coupling (good)	Modules share only the data they need, via parameters	<code>calculateTax(amount, rate)</code>
Message coupling (best)	Modules communicate only through abstract interfaces / messages	<code>gateway.charge(amount)</code> where <code>gateway</code> is an interface

Tightly coupled — Java:

```
java
// OrderService knows EVERYTHING about MySQLDatabase: the class,
// the constructor, the connection string, even SQL syntax.
public class OrderService {
    private MySQLDatabase db = new MySQLDatabase("jdbc:mysql://prod:3306");

    public void placeOrder(Order order) {
        db.execute("INSERT INTO orders VALUES (" + order.getId() + ")");
    }
}
// Switch to Postgres? Test without a real DB? Rewrite OrderService.
```

Loosely coupled — Java:

```
java
```

```

public interface OrderRepository {           // abstraction: a message boundary
    void save(Order order);
}

public class OrderService {
    private final OrderRepository repo;      // knows only the interface
    public OrderService(OrderRepository repo) { this.repo = repo; }

    public void placeOrder(Order order) { repo.save(order); }
}

// MySQL, Postgres, in-memory fake for tests – OrderService never changes.

```

The same idea in Python:

```

python

# Tight: OrderService constructs and commands a concrete DB
class OrderService:
    def __init__(self):
        self.db = MySQLDatabase("mysql://prod:3306") # welded to MySQL

# Loose: dependency passed in; anything with .save() works (duck typing)
class OrderService:
    def __init__(self, repo):
        self.repo = repo

    def place_order(self, order):
        self.repo.save(order)

```

Important: the goal is not *zero* coupling — modules must collaborate, so some coupling is necessary. The goal is coupling to **stable abstractions** instead of **volatile details**. Depending on `List<T>` or a `PaymentGateway` interface is cheap; depending on `StripeClientV2Impl` is expensive.

3.2 Cohesion — how focused a module is internally

Cohesion measures how strongly the elements *inside* a module belong together. High cohesion means everything in the class serves one purpose.

Low cohesion — the "Utils" dumping ground:

```

java

```

```
public class Utils {                                // no theme; a junk drawer
    public static String formatDate(Date d) { ... }
    public static double calculateTax(double amt) { ... }
    public static void sendEmail(String to) { ... }
    public static boolean validateAadhaar(String id) { ... }
}
```

Four unrelated jobs, four unrelated reasons to change, four sets of unrelated dependencies dragged in by anyone who imports it.

High cohesion — one theme per class:

```
python

class InvoiceCalculator:                            # everything here is about invoice math
    def subtotal(self, items): ...
    def tax(self, subtotal, region): ...
    def total(self, items, region): ...

class InvoiceFormatter:                             # everything here is about presentation
    def as_pdf(self, invoice): ...
    def as_html(self, invoice): ...
```

3.3 Why they move together

- Low cohesion **creates** coupling: when a class does five jobs, everyone who needs job #1 gets welded to jobs #2-5 (this is exactly Joe's immobile code).
- High cohesion **enables** low coupling: focused modules expose small, stable surfaces that others can depend on safely.
- SRP (coming later) is really a **cohesion** rule; DIP is really a **coupling** rule. The whole of SOLID lives inside these two words.

Quick self-test for any class you write: Can you describe the class in one sentence without using "and"? If not, cohesion is low. Does changing this class force recompiling/retesting classes that logically shouldn't care? If yes, coupling is high.

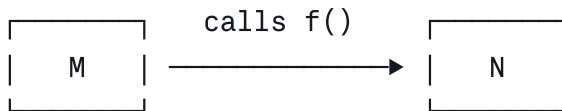
4. Flow of Control vs Source Code Dependency

This is the most important idea in these notes. Everything else is built on it.

4.1 The two arrows

Take two modules. M calls a function $f()$ that lives in N.

FLOW OF CONTROL (runtime)



SOURCE CODE DEPENDENCY (compile time)



Which module *knows about* the other? M knows about N. How does the **compiler** know? Because N's name literally appears inside M's source code:

java

```
// Java
import com.company.N;
```

python

```
# Python
from company import n
```

That import statement **is** the dependency arrow. Its consequences: if N changes, M must be recompiled, redeployed, and retested — and so must everything that depends on M.

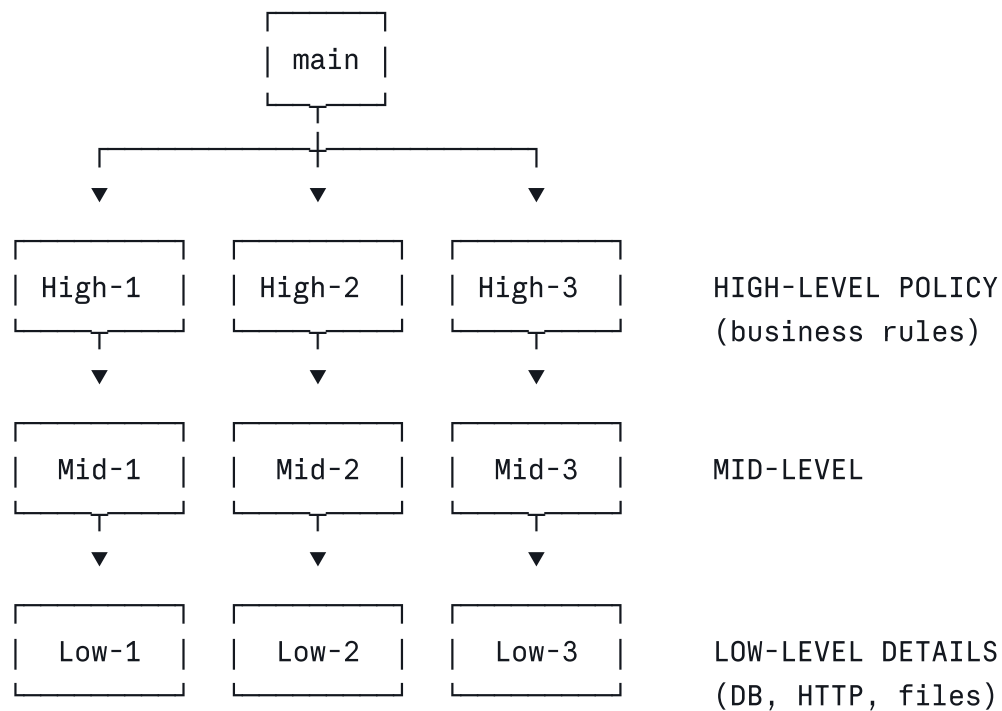
The universal law of procedural code:

```
| The source code dependency ALWAYS points in the |
| SAME direction as the flow of control.          |
|                                                  |
| Caller —imports—> Callee                        |
| Caller —calls—> Callee                          |
```

To call something you must name it; to name it you must depend on it. **You have no choice.** Remember that phrase — "no choice" — it is the setup for the biggest reveal in these notes (Section 6).

4.2 The procedural call tree

Every application, in every language, has this shape:



So in procedural code, **high-level policy depends on low-level details**. Change a tiny variable in a low-level module, and every module up the chain must recompile and redeploy. *A detail changed and high-level policy was affected.* Think about that for any length of time and you'll conclude it's insane.

4.3 A real-world chain: Checkout → Stripe

```
Checkout      ← business policy: "charge the customer"
  | imports
  ▼
StripeClient  ← vendor SDK
  | imports
  ▼
HTTP library → TCP sockets
```

java

```
// Checkout.java – the "before" picture
import com.stripe.StripeClient;           // ← the dependency arrow, in writing

public class Checkout {
    private StripeClient stripe = new StripeClient("sk_live_...");

    public void completeOrder(Order order) {
        stripe.charge(order.getTotal(), order.getCard()); // policy knows Stri
    }
}
```

Should your business policy know that networking exists? Should `Checkout` recompile because Stripe renamed a parameter? This downward-pointing dependency is precisely why code becomes **rigid** (recompile cascades), **fragile** (a low-level change breaks high-level behavior), and **immobile** (policy is welded to details).

"High-level policy should survive when every framework around it changes. If changing a database forces you to rewrite business logic, your architecture is upside down."

For roughly 40 years — Fortran, COBOL, C — programmers had **no way to fix this**. The arrows had to point with the flow of control. Hold that thought.

5. The Three "Magic Words" of OO — An Honest Audit

By the 2000s, every mainstream language was object-oriented. Ask why, and the standard answer is three magic words: **encapsulation**, **inheritance**, **polymorphism**. Let's audit each claim honestly — because the audit's surprising result is what reveals OO's *real* contribution.

5.1 Encapsulation — OO actually made it WORSE ❌

C had perfect encapsulation. Declaration and implementation lived in different files:

```
c
/* point.h – what users see. NOTHING else is visible. */
struct Point;                               /* name only – no members! */
struct Point* makePoint(double x, double y);
double distance(struct Point* p1, struct Point* p2);
```

```
c
/* point.c – the implementation. Users CANNOT see this. */
struct Point { double x; double y; };        /* members hidden here */
```


Users include `point.h` — they see function signatures and the *name* of the struct, but **not a single data member**. Encapsulation enforced by physical file separation.

Then C++ (for compiler-technical reasons — the compiler needed to know the object's *size*) forced member variables into the header:

```
cpp
// point.h - C++ version. Encapsulation BROKEN.
class Point {
public:
    Point(double x, double y);
    double distance(const Point& other) const;
private:
    double x;    // "private"... but sitting RIGHT THERE in the header.
    double y;    // Users see it. Users RECOMPILE when it changes.
};
```

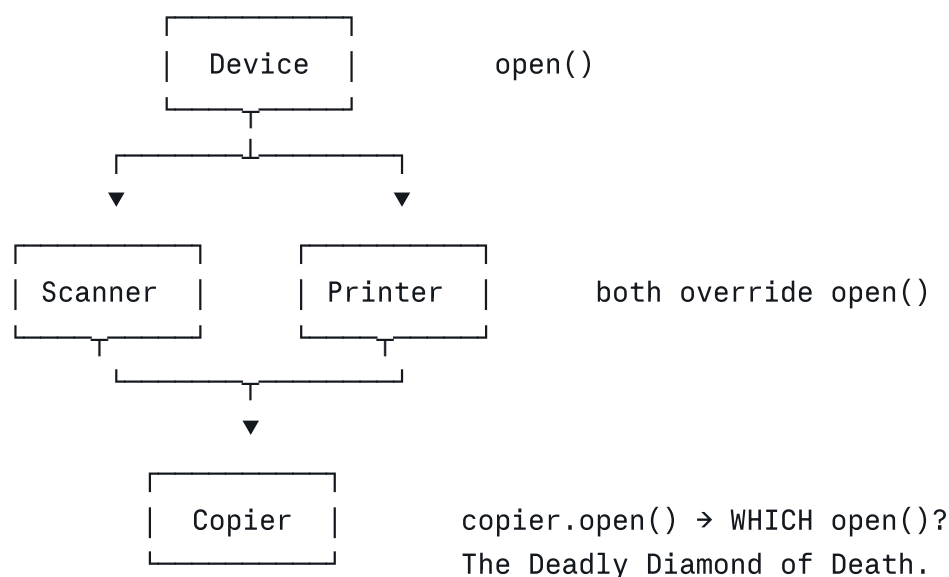
`public` / `private` / `protected` were invented as **band-aids** — compiler-enforced apologies for the encapsulation the language had physically destroyed. (Python doesn't even pretend: `_name` is just a naming convention.)

┃ **Verdict: OO weakened encapsulation. C wins.**

5.2 Inheritance — slightly more convenient 🤖 (half credit)

C programmers faked inheritance for decades by aligning struct layouts and casting pointers. C++ made it convenient and safe. Half credit.

The Diamond Problem



The ambiguity was solved by several languages (C++ via `virtual` inheritance; Python via MRO). **Java refused to solve it** — it banned multiple inheritance of classes and invented the

interface as a workaround: effectively an abstract class with no state, which is safe to multiply-inherit because there's no data to be ambiguous about. C# copied Java.

Python solves it deterministically (Method Resolution Order):

```
python

class Device:
    def open(self): print("device open")

class Scanner(Device):
    def open(self): print("scanner open")

class Printer(Device):
    def open(self): print("printer open")

class Copier(Scanner, Printer):      # multiple inheritance – allowed!
    pass

Copier().open()                    # → "scanner open"  (left-to-right MRO)
print(Copier.__mro__)              # Copier → Scanner → Printer → Device → object
```

Java: abstract class vs interface — what's allowed

Feature	Abstract class	Interface
Instance fields (state)	✓	✗ (only <code>public static final</code> constants)
Constructors	✓	✗
Method implementations	✓	✓ since Java 8 (<code>default</code> methods)
Access modifiers on members	✓ any	methods effectively <code>public</code>
A class can extend / implement	only ONE	MANY

The only special thing about a Java interface is what you **can't** do with it. (Irony: once Java 8 added `default` methods, the diamond returned for *behavior*, and Java had to add tie-breaking rules anyway.)

5.3 Polymorphism — the real prize ✓

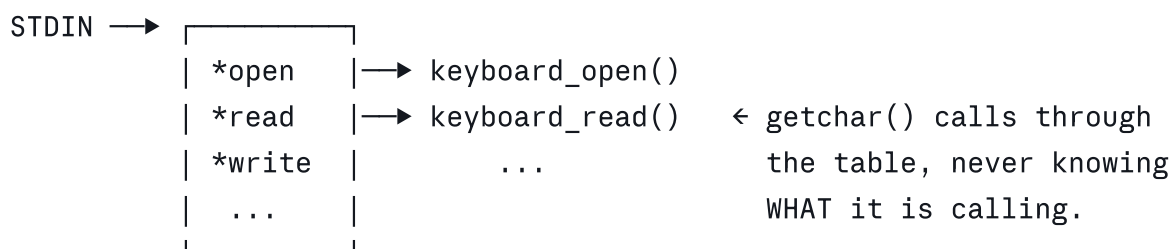
The most important program in these notes — the UNIX `copy` program, ~1971 (shown in C for history, understood in any language):

```
c
```

```
void copy(void) {
    int c;
    while ((c = getchar()) != EOF)
        putchar(c);
}
```

It copies standard input to standard output. But what is standard input? It defaults to the keyboard — but it can be a file, a pipe, a socket, an SSD, a device **not yet invented in 1971**. `getchar` and `putchar` are **polymorphic calls**. This 5-line program still works today with hardware its author never imagined.

How? Every I/O driver implemented 5 functions with identical signatures — `open, close, read, write, seek` — and the OS kept a **table of pointers** to them:



C++ programmers will recognize that table: **it's a vtable**. Polymorphism has always been "just" function pointers in a table.

This also answers a classic interview question: **why does Java have no pointers to functions?** Because polymorphic languages don't need them — **polymorphism IS the safe, compiler-managed use of function pointers**. (Python and JS pass functions around freely, but the same logic holds: polymorphic dispatch removes the need for raw pointer tables.)

So why didn't C programmers use this everywhere? Because hand-maintained pointer tables were **dangerous** — forget to initialize one entry, and you're debugging a crash at 2 AM. The discipline had to be maintained manually.

What OO really gave us: polymorphism that is CHEAP, EASY, and SAFE.

Bonus: dynamic languages get polymorphism WITHOUT inheritance (duck typing)

python

```

class File:
    def read(self): return "from file"

class Socket:                                # NOTE: no common base class!
    def read(self): return "from socket"

class Keyboard:
    def read(self): return "from keyboard"

def copy(source):                             # works with ALL of them
    return source.read()                     # "if it quacks like a duck..."

for src in (File(), Socket(), Keyboard()):
    print(copy(src))

```

In Java you *must* use inheritance or interfaces to get polymorphism — arguably a weakness of static typing, not a feature. In Python, inheritance is only for sharing *implementation*; polymorphism is free.

5.4 The scorecard

Magic word	Did OO invent it?	Verdict
Encapsulation	No — C had it <i>better</i>	✗ OO weakened it
Inheritance	No — C faked it adequately	👉 Half credit (convenience)
Polymorphism	No — C had it via function pointers	✓ OO made it cheap, easy, SAFE

The marketing slogans — "OO models the real world," "OO is closer to how we think" — were invented to sell the idea to executives. If they're wrong, and two of the three magic words are overstated, then **what is OO actually for?** The answer is what *safe polymorphism lets you do* — the next section.

6. Inversion of Control — The Real Reason OO Exists ★

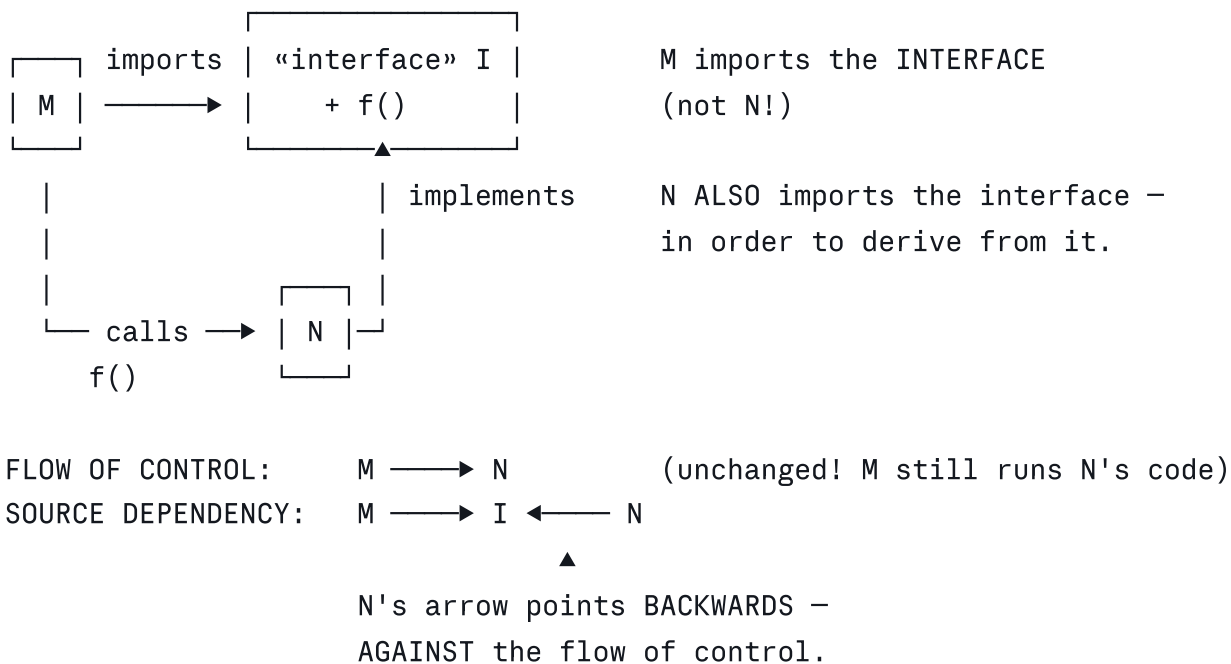
If you remember one section from these notes, make it this one.

6.1 The problem, restated

Section 4 established the "no choice" law: for 40 years, the source-code dependency arrow was **chained** to the flow of control. Callers depended on callees. Policy depended on details. And all three diseases of Section 2 grow from that one fact.

6.2 The trick — insert an abstraction

With cheap, safe polymorphism, you can do something that was impossible before:



Stare at this diagram until both statements feel true at once:

1. **The flow of control did not change.** At runtime, M still ends up executing N's code.
2. **The compile-time dependency reversed.** N now depends on the abstraction that M uses. The arrow flipped.

This is called **dependency inversion**, and the broader idea it enables is **Inversion of Control (IoC)**. For the first time in the history of programming, the dependency structure of a system became **independent** of its call structure. You can take *any* arrow in your architecture and decide — as a design choice — which way it points.

"Absolute control over dependency direction = absolute control over rigidity, fragility, and immobility."

6.3 What "inversion of CONTROL" means

The name comes from *who is in control of the relationship*:

- **Traditional:** the high-level module controls AND depends on the low-level module. Low-level modules are in a position of power — change one, and policy above it shatters.
- **Inverted:** the high-level module **defines an interface it needs** ("here is the shape of the plug"), and low-level modules must **conform to it** ("we are plugs"). Control now belongs to the policy. Details serve; they no longer dictate.

This is nicknamed the **Hollywood Principle**: *"Don't call us, we'll call you."* The framework/policy calls your detail code through an abstraction; your detail code never gets to dictate terms upward.

You already use IoC every day without noticing:

Everyday thing	The inversion
list.sort(key=...)/ Collections.sort(list, comparator)	The library calls <i>your</i> comparison logic — you don't call the sort internals
A web framework routing to your handler function	Framework's main runs the show and calls <i>you</i>
Button onClick handlers	The UI toolkit calls your code when it decides to
JUnit / pytest running your test methods	The runner is in control, your tests are plugins
VS Code extensions, webpack loaders	The host defines interfaces; plugins conform

Notice that a USB port is the physical version of this idea: the laptop (policy) defines the socket (interface); every device (detail) must conform to it. The laptop was built years before your new mouse existed — yet it can "call" the mouse. That's OCP-by-hardware.

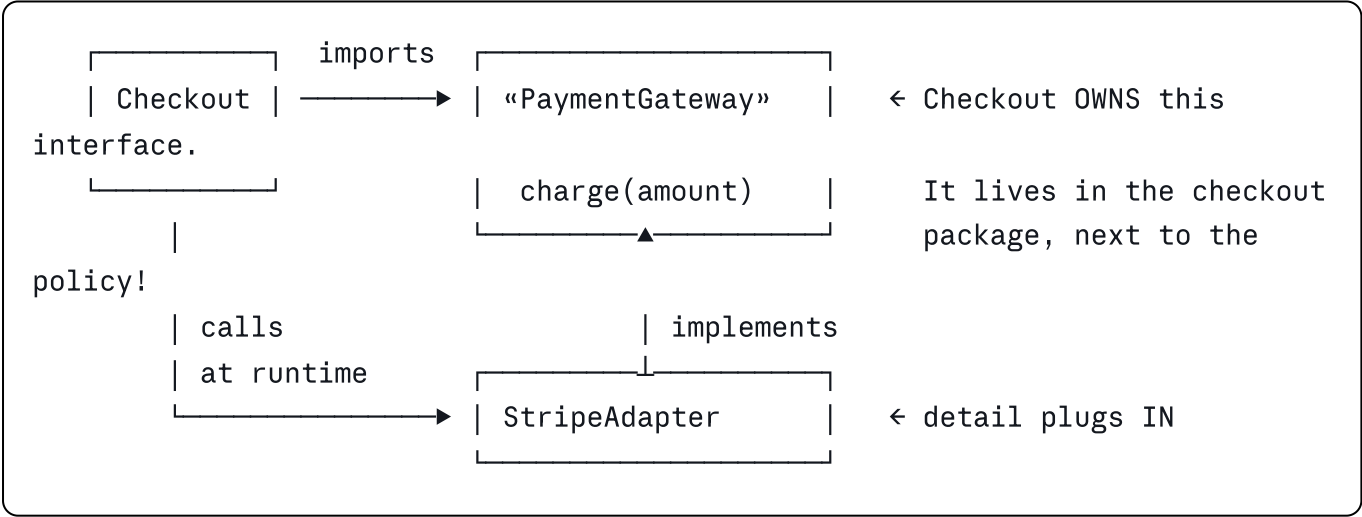
6.4 Worked example 1: Checkout → Stripe, inverted

Before:

Checkout —imports→ StripeClient

 (policy depends on detail ✗)

After:



```
java
```

```
// PaymentGateway.java – owned by the BUSINESS side. Stable.
public interface PaymentGateway {
    Receipt charge(Money amount, PaymentMethod method);
}

// Checkout.java – pure policy. Zero knowledge of Stripe, HTTP, JSON.
public class Checkout {
    private final PaymentGateway gateway;           // depends on abstraction

    public Checkout(PaymentGateway gateway) {       // dependency INJECTED
        this.gateway = gateway;
    }
    public void completeOrder(Order order) {
        gateway.charge(order.total(), order.paymentMethod());
    }
}

// StripeAdapter.java – a detail. Lives at the edge. Replaceable.
public class StripeAdapter implements PaymentGateway { // arrow points BACK a
    private final StripeClient stripe = new StripeClient("sk_live...");
    public Receipt charge(Money amount, PaymentMethod method) {
        return Receipt.from(stripe.charge(amount.cents(), method.token()));
    }
}
```

```
python

# Python – duck typing makes the interface implicit (or use abc.ABC to make it
class Checkout:
    def __init__(self, gateway):                # anything with .charge() works
        self._gateway = gateway
    def complete_order(self, order):
        self._gateway.charge(order.total, order.payment_method)

class StripeAdapter:
    def charge(self, amount, method): ...      # Stripe details live here

class RazorpayAdapter:
    def charge(self, amount, method): ...      # tomorrow's provider – Checkout u

checkout = Checkout(StripeAdapter())          # wiring happens at the edge (main
```

Why this is a big deal — four concrete wins:

1. **Extension without modification:** "Add Razorpay" → write `RazorpayAdapter`.
`Checkout` doesn't change, doesn't recompile, doesn't retest.

2. **Small blast radius:** Stripe changes their SDK → only `StripeAdapter` recompiles. One file.
3. **Testability:** unit-test `Checkout` with a `FakeGateway` — no network, no Stripe account, millisecond tests. (Testability is the *first* practical payoff of IoC you'll feel.)
4. **AI-era bonus:** an AI assistant asked to "add Razorpay support" needs only `PaymentGateway` + one adapter in its context. Tiny context, cheap tokens, safe change.

6.5 Worked example 2: notifications

A second example, because one is never enough. Business rule: "when an order ships, notify the customer."

Naive (policy depends on details):

java

```
public class OrderShippedHandler {
    private EmailClient email = new EmailClient("smtp://...");
    private SmsClient sms = new SmsClient("twilio-key");

    public void handle(Order order) {
        email.send(order.customerEmail(), "Shipped!");
        sms.send(order.customerPhone(), "Shipped!");
        // Boss: "Add WhatsApp." You: edit this class. Again. Forever.
    }
}
```

Inverted:

java


```

public interface Notifier {                                // policy defines the plug
    void notify(Customer customer, String message);
}

public class OrderShippedHandler {
    private final List<Notifier> notifiers;
    public OrderShippedHandler(List<Notifier> notifiers) { this.notifiers = notifiers; }

    public void handle(Order order) {
        for (Notifier n : notifiers)
            n.notify(order.customer(), "Your order has shipped!");
    }
}

public class EmailNotifier implements Notifier { /* SMTP details */ }
public class SmsNotifier implements Notifier { /* Twilio details */ }
public class WhatsAppNotifier implements Notifier { /* NEW — nothing above changes */ }

```

python

```

class OrderShippedHandler:
    def __init__(self, notifiers):
        self._notifiers = notifiers
    def handle(self, order):
        for n in self._notifiers:
            n.notify(order.customer, "Your order has shipped!")

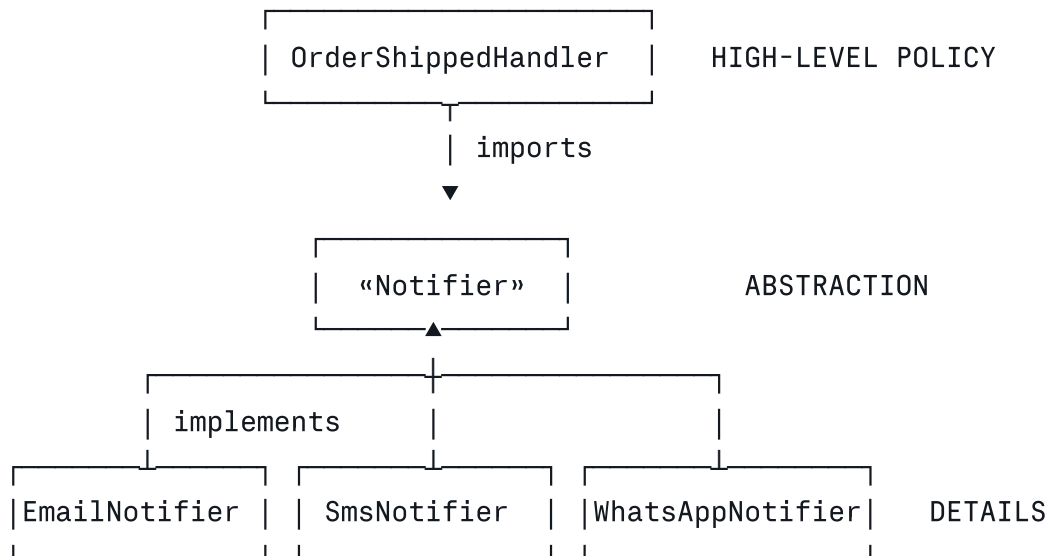
class EmailNotifier:
    def notify(self, customer, message): ...

class SmsNotifier:
    def notify(self, customer, message): ...

handler = OrderShippedHandler([EmailNotifier(), SmsNotifier()]) # wire at the end

```

The dependency graph after inversion — note where every arrow points:



ALL arrows point UP – toward the abstraction, toward stability.
Details depend on policy's contract. Never the reverse.

6.6 IoC vs DI vs DIP — three related terms, untangled

Students (and interviewers) constantly confuse these:

Term	What it is	Analogy
IoC — Inversion of Control	The broad <i>idea</i> : high-level code defines contracts and calls details through them; control flows from policy, not details	The philosophy
DIP — Dependency Inversion Principle	The <i>design rule</i> (the D in SOLID): depend on abstractions, never on concretions; details depend on policy	The law
DI — Dependency Injection	One <i>technique</i> to achieve it: dependencies are handed in from outside (constructor/setter/parameter) instead of constructed inside	The mechanism

Frameworks like **Spring** (Java) and **FastAPI's Depends** (Python) are called *IoC containers* — their whole job is automating the wiring you saw in `checkout = Checkout(StripeAdapter())`. The principle is yours to understand; the container is just a labor-saving device.

6.7 Where this one trick shows up

Polymorphic dependency inversion

- |
- |→ SOLID (especially D)
- |→ Dependency Injection frameworks (Spring, Guice, FastAPI Depends)
- |→ Plugin architectures (VS Code extensions, webpack loaders, JDBC drivers)
- |→ Hexagonal Architecture / Ports & Adapters
- |→ Clean Architecture ("all dependencies point inward")
- |→ Event-driven systems & the Observer pattern
- |→ Microservice contracts (consumer-driven)

6.8 The definition to memorize

"OO is about managing dependencies by selectively **INVERTING** key dependencies in your architecture, so that you can prevent rigidity, fragility, and non-reusability."

Not "modeling the real world." Not "closer to how we think." **Dependency control.**

"Classes don't make software flexible. Dependency direction does."

7. The SOLID Principles

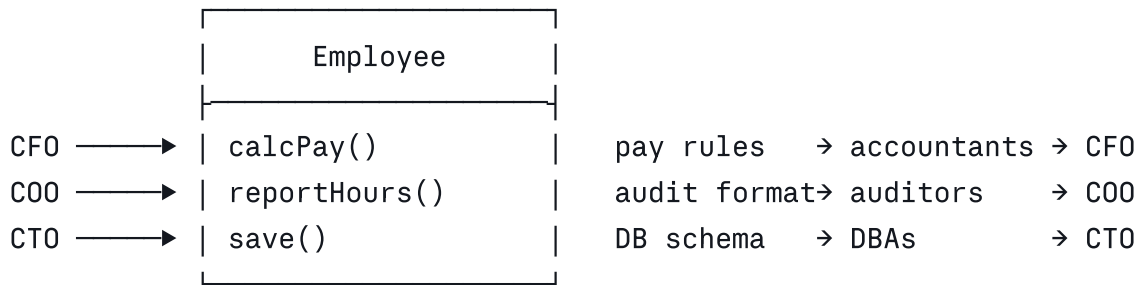
Now that you know *why* — the five principles are just five disciplined ways to control dependencies (coupling) and organize responsibilities (cohesion).

S — Single Responsibility	one reason to change	(a cohesion rule)
O — Open-Closed	extend without modifying	(a coupling rule)
L — Liskov Substitution	derivatives must substitute	(a contract rule)
I — Interface Segregation	no forced dependencies	(a coupling rule)
D — Dependency Inversion	policy never depends on detail	(THE coupling rule)

7.1 S — Single Responsibility Principle

"A class should have only **ONE** reason to change." Responsibility ≠ "what it does."
Responsibility = **which actor can demand a change.**

The classic trap — the Employee class



THREE actors. THREE reasons to change. ONE class. 💣

Why it explodes: an auditor asks to change how fractional hours print. A developer edits `reportHours()`... and tweaks a shared rounding helper also used by `calcPay()`. Payroll is now wrong; the CFO discovers it via a million-dollar hole in the books. A *report-format tweak broke payroll* because three masters were mixed in one class.

Before / after in Java

java

```
// VIOLATION — three concerns, three masters
public class Employee {
    public Money calcPay() { ... }      // business rules
    public String reportHours() { ... } // presentation
    public void save() { ... }         // persistence
}
```

java

```
// FIX — one class per reason to change (a facade keeps the old API if needed)
public class PayCalculator { public Money calcPay(Employee e) { ... } }
public class HourReporter { public String reportHours(Employee e) { ... } }
public class EmployeeRepository { public void save(Employee e) { ... } }
```

The same smell in Python

python

```
# VIOLATION – a "god class"
class Report:
    def compute_stats(self, data): ...    # calculation
    def render_html(self, stats): ...    # formatting
    def save_to_db(self, stats): ...    # persistence
    def email_to_boss(self, html): ...    # delivery

# FIX – four cohesive classes
class StatsCalculator: ...
class HtmlRenderer: ...
class ReportRepository: ...
class ReportMailer: ...
```

Rules of thumb: calculation in one place, formatting in another, persistence in another. If you can't describe the class in one sentence without "and", split it. SRP is the cohesion principle.

7.2 O — Open-Closed Principle (Bertrand Meyer, 1988)

"Software entities should be open for extension but closed for modification." Add features by adding NEW code — not by editing OLD code.

Sounds impossible — until you remember the UNIX `(copy)` program: it gained the ability to read from devices invented *decades later*, without ever being recompiled.

The violating version — the switch-statement disease

```
java

// Java
public double area(Shape s) {
    switch (s.getType()) {
        case CIRCLE: return Math.PI * s.r * s.r;
        case SQUARE: return s.side * s.side;
        default: throw new IllegalArgumentException();
    }
}
```

```
python

# Python – same disease, different syntax
def area(shape):
    if shape.type == "circle":
        return 3.14159 * shape.r ** 2
    elif shape.type == "square":
        return shape.side ** 2
```

What happens when you add `Oval`?

1. The shared `type` enum changes → **everything touching shapes recompiles/redeploys** — even `Circle` and `Square`, which couldn't care less about ovals. That's rigidity.
2. It's never *one* switch. There's one for `area`, one for `draw`, one for `rotate`, one for `scale`... some disguised as if/else chains, some "logically optimized" (rotate has no case for circle — rotating a circle does nothing). You **will** miss one. That's fragility.
3. Worst case: the business wants to sell shapes as separate plugins and give the engine away free. Impossible — the switch **welds** the engine to every shape. That's immobility.

The compliant version

```
java

public abstract class Shape { public abstract double area(); }

public class Circle extends Shape {
    private double r;
    public double area() { return Math.PI * r * r; }
}

public class Square extends Shape {
    private double side;
    public double area() { return side * side; }
}

public class Oval extends Shape {           // NEW — nothing else recompiles
    public double area() { ... }
}

public double totalArea(List<Shape> shapes) {
    return shapes.stream().mapToDouble(Shape::area).sum();    // no switch, ever
}
```

python

```

from abc import ABC, abstractmethod

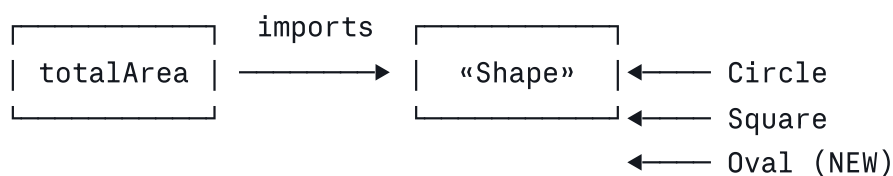
class Shape(ABC):
    @abstractmethod
    def area(self): ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

class Square(Shape):
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2

def total_area(shapes):
    return sum(s.area() for s in shapes)  # closed for modification, open for

```



High-level code has NO IDEA which shapes exist.

⚠ The honest caveat ("the lie")

Customers won't ask for a new shape. They'll ask for *"draw all squares before all circles"* — an **ordering** change your beautiful abstraction doesn't protect against. Customers are mysteriously brilliant at requesting exactly the feature that thwarts your design. So don't try to predict every axis of change up front:

1. Build the **simplest** thing that works.
2. Ship it **early**.
3. Watch **where change requests actually arrive** — that reveals the true axis of change.
4. THEN build the abstraction that protects that axis. **Abstractions are earned by evidence, not guessed in advance.**

7.3 L — Liskov Substitution Principle (Barbara Liskov, 1988)

"Derived classes must be usable through the base class interface without the user needing to know the difference."

The dreaded Rectangle/Square

Geometry says a square IS-A rectangle. So:

python

```

class Rectangle:
    def set_width(self, w): self.width = w
    def set_height(self, h): self.height = h
    def area(self): return self.width * self.height

class Square(Rectangle):          # "a square is a rectangle", right?
    def set_width(self, w):      # must keep sides equal...
        self.width = self.height = w
    def set_height(self, h):
        self.width = self.height = h

```

Now an innocent client, written purely against `Rectangle`:

```

python

def stretch(rect):
    rect.set_width(5)
    rect.set_height(4)
    assert rect.area() == 20      # a Rectangle user's RIGHT to expect...
                                # ✨ Square returns 16. Corrupted state,
                                # discovered a billion instructions later.

```

The fix people reach for makes it worse:

```

python

def stretch(rect):
    if isinstance(rect, Square): # ← the smell. You now DEPEND on Square –
        ...                     # a class you never wanted to know about.

```

Every `isinstance` (Python) / `instanceof` (Java) type-check like this is an OCP violation growing out of an LSP violation: the client is now coupled to a concrete subclass, and every new subclass means editing the client again.

A second example — Bird/Penguin (Java)

```

java

```



```
// VIOLATION
public class Bird {
    public void fly() { ... }
}
public class Penguin extends Bird {
    @Override public void fly() {
        throw new UnsupportedOperationException("Penguins can't fly"); // ✨
    }
}
// Any code that loops over List<Bird> and calls fly() now explodes
// at runtime – but only when a penguin happens to be in the list.
```

```
java
// FIX – model the CAPABILITY, not the taxonomy
public abstract class Bird { }
public interface Flying { void fly(); }

public class Sparrow extends Bird implements Flying {
    public void fly() { ... }
}
public class Penguin extends Bird { } // simply not Flying

public void launchAll(List<Flying> flyers) { // can never receive a penguin –
    for (Flying f : flyers) f.fly(); // the COMPILER enforces LSP here
}
```

The resolution — the lawyer line

The *geometric* square is a rectangle. But `class Square` is not a square — it is **code that represents** a square, and **representatives do not share the relationships of the things they represent**. (Your divorce lawyer and your spouse's divorce lawyer are not, themselves, getting divorced.)

Inheritance is not "IS-A" — that vocabulary leaked in from 1980s AI research. Inheritance is just *redeclaration of functions and variables in a sub-scope*. The real test is **substitutability**:

- A subclass may not **strengthen preconditions** (demand more than the base promised to demand).
- A subclass may not **weaken postconditions** (deliver less than the base promised to deliver — `Square` broke the postcondition "setting height leaves width unchanged").
- A subclass may not throw surprises the base's clients weren't told to expect (`Penguin.fly()`).

Rule of thumb: if you feel the urge to write `isinstance`/`instanceof` on a hierarchy you own, the hierarchy is wrong.

7.4 I — Interface Segregation Principle

"No client should be forced to depend on methods it does not use." Fat interfaces create phantom coupling: clients get chained to methods they never call — and get recompiled/broken by changes to those methods.

The violating version — the fat interface

```
java

// One interface tries to serve every kind of worker
public interface Worker {
    void work();
    void eat();
    void sleep();
}

public class HumanWorker implements Worker {
    public void work() { ... }
    public void eat() { ... }
    public void sleep() { ... }
}

public class RobotWorker implements Worker {
    public void work() { ... }
    public void eat() { throw new UnsupportedOperationException(); } // ✨ r
    public void sleep() { throw new UnsupportedOperationException(); } // (not
                                                                    // of
```

Two problems:

1. `RobotWorker` is **forced** to implement methods that make no sense for it — and the least-bad option (throwing) breaks LSP.
2. A scheduler class that only needs `work()` still *depends on* `eat()` and `sleep()`. If the `eat()` signature changes (say, `eat(MealPlan plan)`), the scheduler recompiles — for a method it never calls. **Phantom coupling.**

The fix — split by client need

```
java
```

```

public interface Workable { void work(); }
public interface Feedable { void eat(); }
public interface Sleepable { void sleep(); }

public class HumanWorker implements Workable, Feedable, Sleepable { ... }
public class RobotWorker implements Workable { ... }           // only what

// Each client depends ONLY on the capability it uses:
public class Scheduler {
    public void assign(Workable w) { w.work(); }           // untouched by eat()/sleep()
}
public class Cafeteria {
    public void serve(Feedable f) { f.eat(); }
}

```

The same idea in Python — the multifunction printer

```

python

# VIOLATION — every device forced to pretend it's a full MFP
class Machine(ABC):
    @abstractmethod
    def print_doc(self, d): ...
    @abstractmethod
    def scan_doc(self, d): ...
    @abstractmethod
    def fax_doc(self, d): ...

class OldPrinter(Machine):
    def print_doc(self, d): ...
    def scan_doc(self, d): raise NotImplementedError # ✖
    def fax_doc(self, d): raise NotImplementedError # ✖

```

```
python
```

```
# FIX – small, role-based interfaces (Python calls these "protocols")
class Printer(ABC):
    @abstractmethod
    def print_doc(self, d): ...

class Scanner(ABC):
    @abstractmethod
    def scan_doc(self, d): ...

class OldPrinter(Printer):
    # implements only what it can honestly
    def print_doc(self, d): ...

class MultiFunctionPrinter(Printer, Scanner):
    def print_doc(self, d): ...
    def scan_doc(self, d): ...
```

How to spot ISP violations in the wild:

- Implementations full of `throw new UnsupportedOperationException()` / `raise NotImplementedError` / empty method bodies.
- Interfaces named after *things* (`Machine`, `Worker`, `Manager`) instead of *capabilities* (`Workable`, `Printable`, `Closeable`).
- Clients importing an interface but calling only 1 of its 12 methods.

Rule of thumb: design interfaces from the **client's** point of view, not the implementer's.

Many small role-interfaces beat one fat entity-interface. (Notice Java's own library learned this: `Comparable`, `Iterable`, `Closeable`, `Runnable` — tiny, capability-shaped, one or two methods each.)

7.5 D — Dependency Inversion Principle

1. High-level modules should not depend on low-level modules. Both should depend on abstractions. 2. Abstractions should not depend on details. Details should depend on abstractions.

This is Section 6, formalized — the capstone principle, the one the other four exist to serve. Two extra points that Section 6 didn't cover:

Who owns the abstraction? (the most-missed detail)

The interface belongs to the **high-level side** — it lives in the *policy's* package, and it is named in the *policy's* language:

× WRONG ownership: stripe_integration/ package IStripeClient ← Checkout "port" (interface shaped like the vendor; policy still speaks "Stripe") "adapter"	✓ RIGHT ownership: checkout/ ← policy Checkout PaymentGateway ← the adapters/ StripeAdapter — the
---	---

If your interface is called (`IStripeClient`) with methods mirroring Stripe's API, you haven't inverted anything — you've just added a layer of indirection around the same dependency. The interface must express **what the policy needs** (`charge(amount)`), not **what the vendor offers** (`createPaymentIntent(intentParams)`). This is exactly the "Ports & Adapters" in Hexagonal Architecture.

A complete before/after — the weather app

```
java
// VIOLATION - Java
public class WeatherReporter {                                // high-level policy
    private OpenWeatherApi api = new OpenWeatherApi("key");    // low-level detail
                                                                // + `new` INSIDE the clas
    public String report(String city) {                          // = hard-wired dependen
        JsonNode json = api.fetchRaw(city);                      // policy parses vendor JS
        return "Temp: " + json.get("main").get("temp");
    }
}
```

```
java
```

```
// FIX – Java
public interface WeatherSource {                                // abstraction, owned by p
    Weather current(String city);                               // speaks the POLICY's lan
}

public class WeatherReporter {
    private final WeatherSource source;
    public WeatherReporter(WeatherSource source) { this.source = source; } //

    public String report(String city) {
        return "Temp: " + source.current(city).temperature();
    }
}

public class OpenWeatherAdapter implements WeatherSource {    // detail, at the
    public Weather current(String city) { /* API call + JSON parsing here */ }
}

// main() – the ONLY place that knows concrete types. All the `new` lives here.
WeatherReporter reporter = new WeatherReporter(new OpenWeatherAdapter());
```

python

```
# FIX – Python (with an explicit abstraction via abc)
from abc import ABC, abstractmethod

class WeatherSource(ABC):
    @abstractmethod
    def current(self, city): ...

class WeatherReporter:
    def __init__(self, source: WeatherSource):
        self._source = source
    def report(self, city):
        return f"Temp: {self._source.current(city).temperature}"

class OpenWeatherAdapter(WeatherSource):
    def current(self, city): ...          # requests + JSON parsing here

class FakeWeather(WeatherSource):        # tests become trivial
    def current(self, city): return Weather(temperature=25)

reporter = WeatherReporter(OpenWeatherAdapter())    # composition root
```

How to spot DIP violations in the wild

- `(new SomeConcreteDetail(...))` **inside** business-logic classes (construction is a dependency!).
- `(import stripe, import com.mysql...)` statements in policy code that name vendors, drivers, or frameworks (`(import stripe, import com.mysql...)`).
- Business classes that are impossible to unit-test without a network, database, or API key.
- Interfaces that mirror a vendor's API instead of the policy's needs.

Rule of thumb: all the `(new)` / concrete construction should be squeezed into one place at the edge of the system (the *composition root* — usually `(main)` or a DI container like Spring). Everything inward of that speaks only in abstractions.

8. Summary

1. Bad code has three names — **rigidity, fragility, immobility** — and one cause: **coupling**.
2. The design goal, always: **low coupling, high cohesion**.
3. In procedural code, dependency arrows are chained to the flow of control: policy is forced to depend on details.
4. OO's honest contribution is not encapsulation (weakened it) or inheritance (half credit) — it is **cheap, easy, SAFE polymorphism**.
5. Safe polymorphism enables **Inversion of Control**: insert an abstraction and the dependency arrow flips *against* the flow of control. Policy defines contracts; details conform. "Don't call us, we'll call you."
6. **That is why OO exists**: selectively inverting key dependencies to prevent rigidity, fragility, and non-reusability.
7. SOLID = five disciplines for doing this well. S is a cohesion rule; O, I, D are coupling rules; L keeps the abstractions honest.
8. In the AI era, machines type the code — but **someone must decide where code lives and which way the arrows point**. That is the job.